Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Program: Software Engineering

Operating Systems

Laboratory Work No. 6

Inside the xv6 File System

Track B: Build Your Own Small OS (xv6)

Elaborated by:

st. gr. FAF-241
Luchiciov Alexei

Verified by:

Reiţman Patricia

Chișinău – 2026

Contents

1. Introduction....3
2. Reading the xv6 File System....3
  2.1. Observations....4
  2.2. Interesting Command Output....4
3. The mystat Program and Hard Links....4
  3.1. Implementation....5
  3.2. Hard-Link Experiment....5
  3.3. Observations....6
4. Measuring File-System Limits....6
  4.1. Constants and Disk Layout....6
  4.2. Observations....7
5. Recursive find Program....7
  5.1. Implementation....7
  5.2. Testing....8
  5.3. Observations....8
6. Conclusion....9

1. Introduction

The purpose of this laboratory work was to study the internal organization of the xv6 file system. The work focused on file types, inode numbers, directory entries, hard links, file-system limits, and the structure of an on-disk inode. I also implemented two xv6 user programs: mystat, which prints metadata returned by stat(), and find, which recursively searches a directory tree.

The experiments were performed in the existing ~/xv6-riscv source tree inside the Ubuntu virtual machine. The xv6 system was rebuilt and launched in QEMU when tests inside the teaching operating system were required.

2. Reading the xv6 File System

I first rebuilt xv6 from a clean state and inspected the output of the xv6 ls program. In xv6 each line contains a name, type, inode number, and size in bytes:

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
...
console        3 25 0

$ mkdir a
$ ls a
.              1 26 32
..             1 1 1024
```

The definitions in kernel/stat.h were then inspected:

```
#define T_DIR    1 // Directory
#define T_FILE   2 // File
#define T_DEVICE 3 // Device
```

2.1. Observations

1. Type 1 means a directory, type 2 means a regular file, and type 3 means a device. This matches the definitions in kernel/stat.h.

2. The root directory has inode number 1. In the root directory both . and .. refer to inode 1 because the root is its own parent.

3. The new empty directory a had size 32 bytes and contained only . and ... Therefore one directory entry occupies 16 bytes. This also agrees with the xv6 directory-entry layout: an inode number plus a fixed-size name field.

2.2. Interesting Command Output

The most useful output in this part was the listing of the empty directory. It showed directly that . points to the new directory inode while .. points back to the root inode:

```
$ ls a
.              1 26 32
..             1 1 1024
```

3. The mystat Program and Hard Links

The next task was to write a small user program named mystat. It calls stat() for every path supplied on the command line and prints the file type, inode number, link count, and size.

3.1. Implementation

```
char *
typename(int t)
{
    if(t == T_DIR) return "directory";
    if(t == T_FILE) return "file";
    if(t == T_DEVICE) return "device";
    return "?";
}

for(int i = 1; i < argc; i++){
    if(stat(argv[i], &st) < 0){
        fprintf(2, "mystat: cannot stat %s\n", argv[i]);
        continue;
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), st.ino,
           st.nlink, (int)st.size);
}
```

The program was added to UPROGS in the Makefile as $U/_mystat\, after which xv6 was rebuilt successfully.

3.2. Hard-Link Experiment

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 1, size 1024 bytes
console: device, inode 26, links 1, size 0 bytes

$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes

$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes

$ cat r2 | wc
48 336 2441
```

Creating r2 did not copy the file. Instead, both names referred to the same inode 2 and the link count increased from 1 to 2. Removing the name README reduced the link count back to 1, while the contents remained accessible through r2.

3.3. Observations

1. After ln README r2, README and r2 had the same inode number (2), the same size (2441 bytes), and a link count of 2. After rm README, r2 still had inode 2 and the same contents, but its link count became 1.

2. In kernel/sysfile.c, sys_unlink starts at line 202 in my source tree. The ordinary file link count is decremented at line 244 with ip->nlink--. The data blocks are not freed merely because one directory name is removed; they can be released when the inode has no remaining links and no remaining references.

3. The on-disk struct dinode in kernel/fs.h contains type, major, minor, nlink, size, and addrs[NDIRECT + 1]. Unlike a full Linux inode, this xv6 inode does not contain permission bits or symbolic-link metadata.

```
struct dinode {
  short type;
  short major;
  short minor;
  short nlink;
  uint size;
  uint addrs[NDIRECT + 1];
};
```

4. Measuring File-System Limits

4.1. Constants and Disk Layout

The relevant constants were read directly from kernel/fs.h:

```
#define BSIZE      1024
#define NDIRECT      12
#define NINDIRECT (BSIZE / sizeof(uint))
#define DIRSIZ       14
```

Because a uint block number occupies 4 bytes, the indirect block can store 1024 / 4 = 256 block addresses. The largest file supported by this xv6 inode format is therefore:

```
NINDIRECT = 1024 / 4 = 256 blocks
Maximum blocks = 12 + 256 = 268
Maximum size = 268 * 1024 = 274432 bytes = 268 KiB
```

The file-system image build printed the following layout:

```
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1)
blocks 1953 total 2000
```

Thus the image contains 2000 blocks in total. There are 47 metadata blocks and 1953 data blocks. The metadata includes 31 log blocks, 13 inode blocks, one bitmap block, one boot block, and one superblock.

4.2. Observations

1. DIRSIZ is 14, so an xv6 directory entry reserves at most 14 bytes for the stored name. In my captured run, the command mkdir averyveryverylongname returned "exec mkdir failed", so this particular truncation experiment did not produce an observed directory entry. The 14-byte limit itself is directly visible in kernel/fs.h.

2. The maximum file size in this xv6 version is 274432 bytes (268 KiB): 12 direct blocks plus 256 blocks addressed through one indirect block, with each block being 1024 bytes.

3. The complete disk image contains 2000 blocks. The build reported 31 log blocks, 13 inode blocks, 1 bitmap block, and 1953 data blocks. Together with the boot and super blocks, the metadata total is 47 blocks.

5. Recursive find Program

The final task was to write a find program for xv6. The program receives a starting directory and a target name, reads directory entries recursively, skips free entries and the special . and .. entries, and prints every matching path.

5.1. Implementation

```
while(read(fd, &de, sizeof(de)) == sizeof(de)){
    if(de.inum == 0)
        continue;

    memmove(name, de.name, DIRSIZ);
    name[DIRSIZ] = 0;

    if(strcmp(name, ".") == 0 || strcmp(name, "..") == 0)
        continue;

    strcpy(p, name);

    if(stat(buf, &st) < 0){
        fprintf(2, "find: cannot stat %s\n", buf);
        continue;
    }

    if(strcmp(name, target) == 0)
        printf("%s\n", buf);

    if(st.type == T_DIR)
        find(buf, target);
}
close(fd);
```

The program closes every directory that it opens and was added to UPROGS as $U/_find\. The implementation was based on the directory-reading pattern already used by user/ls.c.

5.2. Testing

After a clean rebuild, I created a small directory tree and three paths named r2:

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2
```

This main recursive test succeeded and showed that find could descend into nested directories and print all matching paths.

The second requested test behaved differently in the captured run:

```
$ find a b
exec find failed

$ find
usage: find <dir> <name>
```

The first recursive test proves that the program was built and executed correctly. The later no-argument invocation also reached the program and printed its usage message. I kept the unexpected exec failure from the second invocation in the report rather than replacing it with an output that was not observed.

5.3. Observations

1. The recursive search works by opening a directory, reading struct dirent entries, constructing a full path, calling stat(), printing a match, and recursively processing entries whose type is T_DIR.
2. Entries with inum == 0 are skipped because they are unused directory slots. The names . and .. are skipped so that recursion does not loop forever between a directory and its parent.
3. The code closes every opened directory with close(fd). This is important because xv6 allows only a small number of open files per process.

6. Conclusion

In this laboratory work I examined the xv6 file system from both the user and kernel sides. I identified the three xv6 file types, observed inode numbers and directory-entry sizes, and used a custom mystat program to see how hard links share the same inode and change the link count.

I also measured important file-system limits directly from kernel/fs.h. In this version, the block size is 1024 bytes, directory names use a 14-byte field, and the inode can address a maximum file of 268 KiB. The generated file-system image contains 2000 blocks, of which 1953 are data blocks.

Finally, I implemented a recursive find program and successfully used it to locate three files named r2 at different levels of the directory tree. This lab made the relationship between directory names, inodes, data blocks, and file-system system calls much clearer.